

OSSP-SKILL-WEEK5

SESSION-9

TASK1: Implement directory navigation in a shell. The program should change directories, validate paths, update the working directory, handle errors, maintain the previous directory, and test navigation.

SOURCE CODE:

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

int main()
{
    char command[100];
    char path[PATH_MAX];
    char previous[PATH_MAX];
    char current[PATH_MAX];

    getcwd(current, sizeof(current));
    strcpy(previous, current);

    printf("Session 9 - Directory Navigation\n");
    printf("-----\n");
    printf("Commands: pwd, cd <path>, prev, exit\n\n");

    while (1)
    {
        getcwd(current, sizeof(current));
        printf("myshell[%s]> ", current);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }
        else if (strcmp(command, "pwd") == 0)
        {
            printf("Current directory: %s\n", current);
        }
        else if (strncmp(command, "cd ", 3) == 0)
        {
            strcpy(path, command + 3);

            if (chdir(path) == 0)
            {
                strcpy(previous, current);
            }
        }
    }
}
```

```

{
    strcpy(path, command + 3);

    if (chdir(path) == 0)
    {
        strcpy(previous, current);
        getcwd(current, sizeof(current));

        printf("Directory changed successfully.\n");
        printf("New directory: %s\n", current);
    }
    else
    {
        printf("Error: Cannot change directory to '%s'\n", path);
        printf("Reason: %s\n", strerror(errno));
    }
}
else if (strcmp(command, "prev") == 0)
{
    if (chdir(previous) == 0)
    {
        char temp[PATH_MAX];

        strcpy(temp, current);
        strcpy(current, previous);
        strcpy(previous, temp);

        printf("Returned to previous directory.\n");
        printf("Current directory: %s\n", current);
    }
    else
    {
        printf("Error: Previous directory is not available.\n");
    }
}
else
{
    printf("Invalid command.\n");
}
}

printf("\nDirectory navigation program terminated.\n");

return 0;
}

```

Observation: I learned how to implement directory navigation in a shell using `chdir()` and `getcwd()`. I understood how to validate directory paths, handle errors, maintain the previous working directory, and test navigation commands.

Task2: Implement built-in command handling in a shell. The program should identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain shell state, and test the dispatch logic.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

void builtin_pwd()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current directory: %s\n", cwd);
}

void builtin_help()
{
    printf("Built-in commands:\n");
    printf("  pwd  - Show current directory\n");
    printf("  help - Show available commands\n");
    printf("  exit - Exit shell\n");
}

void builtin_exit()
{
    printf("Exiting shell...\n");
}

typedef void (*command_function)();

struct command
{
    char name[20];
    command_function function;
};

int main()
{
    char input[100];

    struct command dispatch_table[] =
    {
        {"pwd", builtin_pwd},
        {"help", builtin_help},
        {"exit", builtin_exit}
    };

    int command_count =

```

```

int command_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);

printf("Session 9 - Built-in Command Dispatch\n");
printf("-----\n");
printf("Available commands: pwd, help, exit\n\n");

while (1)
{
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    int found = 0;

    for (int i = 0; i < command_count; i++)
    {
        if (strcmp(input, dispatch_table[i].name) == 0)
        {
            dispatch_table[i].function();
            found = 1;

            if (strcmp(input, "exit") == 0)
                return 0;

            break;
        }
    }

    if (!found)
    {
        printf("Invalid command: %s\n", input);
    }
}

return 0;
}

```

Output:

```

Directory navigation program terminated.
root@LAPTOP-1COJKKG6:~# nano session9_task2.c
root@LAPTOP-1COJKKG6:~# gcc session9_task2.c -o session9_task2
root@LAPTOP-1COJKKG6:~# ./session9_task2
Session 9 - Built-in Command Dispatch
-----
Available commands: pwd, help, exit

myshell> pwd
Current directory: /root
myshell> help
Built-in commands:
  pwd   - Show current directory
  help  - Show available commands
  exit  - Exit shell
myshell> exit
Exiting shell...
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how to identify and execute built-in shell commands using a dispatch table. I understood how function pointers can connect commands with their corresponding functions and how invalid commands can be detected and handled.

Session10

Task1: Implement exit handling in a shell. The program should retrieve the current directory, display the path, process exit requests, clean up resources, save the shell state, and verify correct behaviour.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main()
{
    char command[100];
    char current_dir[PATH_MAX];

    printf("Session 10 - Exit Handling\n");
    printf("-----\n");

    if (getcwd(current_dir, sizeof(current_dir)) != NULL)
        printf("Starting directory: %s\n", current_dir);

    printf("Type 'pwd' to display directory or 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "pwd") == 0)
        {
            if (getcwd(current_dir, sizeof(current_dir)) != NULL)
                printf("Current directory: %s\n", current_dir);
        }
        else if (strcmp(command, "exit") == 0)
        {
            printf("Exit request received.\n");
            printf("Saving shell state...\n");
            printf("Cleaning up resources...\n");
            printf("Shell exited successfully.\n");
            break;
        }
        else
        {
            printf("Invalid command.\n");
        }
    }
}
```

```
    }
    return 0;
}
```

Output:

```
root@LAPTOP-1COJKKG6:~# nano session10_task1.c
root@LAPTOP-1COJKKG6:~# gcc session10_task1.c -o session10_task1
root@LAPTOP-1COJKKG6:~# ./session10_task1
Session 10 - Exit Handling
-----
Starting directory: /root
Type 'pwd' to display directory or 'exit' to quit.

myshell> pwd
Current directory: /root
myshell> exit
Exit request received.
Saving shell state...
Cleaning up resources...
Shell exited successfully.
root@LAPTOP-1COJKKG6:~#
```

Observation: I learned how to retrieve and display the current working directory using `getcwd()`. I also learned how to process an exit request, maintain shell state, clean up resources, and terminate the shell correctly.

Task2: Implement environment variable export functionality in a shell. The program should parse export syntax, update environment variables, validate names, handle existing variables, export to child processes, and test functionality.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

int valid_name(char *name)
{
    if (name[0] == '\0')
        return 0;

    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main()
{
    char command[200];

    printf("Session 10 - Environment Variable Export\n");
    printf("-----\n");
    printf("Use: export NAME=value\n");
    printf("Use: print NAME\n");
    printf("Use: child NAME\n");
    printf("Use: exit\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
```

```

GNU nano 7.2
if (strcmp(command, "exit") == 0)
{
    printf("Shell exiting...\n");
    break;
}

else if (strncmp(command, "export ", 7) == 0)
{
    char *assignment = command + 7;
    char *equal = strchr(assignment, '=');

    if (equal == NULL)
    {
        printf("Invalid export syntax.\n");
        continue;
    }

    *equal = '\0';

    char *name = assignment;
    char *value = equal + 1;

    if (!valid_name(name))
    {
        printf("Invalid variable name: %s\n", name);
        continue;
    }

    if (setenv(name, value, 1) == 0)
        printf("Variable exported: %s=%s\n", name, value);
    else
        perror("setenv");
}

else if (strncmp(command, "print ", 6) == 0)
{
    char *name = command + 6;
    char *value = getenv(name);

    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("Variable not found.\n");
}

```

```

else if (strncmp(command, "child ", 6) == 0)
{
    char *name = command + 6;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
    }
    else if (pid == 0)
    {
        char *value = getenv(name);

        if (value != NULL)
            printf("Child process: %s=%s\n", name, value);
        else
            printf("Child process: variable not found.\n");

        exit(0);
    }
    else
    {
        wait(NULL);
    }
}

else
{
    printf("Invalid command.\n");
}

return 0;
}

```

Output:

```
root@LAPTOP-1COJKKG6:~# nano session10_task2.c
root@LAPTOP-1COJKKG6:~# gcc session10_task2.c -o session10_task2
root@LAPTOP-1COJKKG6:~# ./session10_task2
Session 10 - Environment Variable Export
-----
Use: export NAME=value
Use: print NAME
Use: child NAME
Use: exit

myshell> export NAME=usha
Variable exported: NAME=usha
myshell> print NAME
NAME=usha
myshell> child NAME
Child process: NAME=usha
myshell> |
```

Observation: I learned how to implement environment variable export functionality using `setenv()` and `getenv()`. I understood how to parse and validate variable names, update existing variables, and make environment variables available to child processes created using `fork()`.